Práctica 0

Haga una tabla en la que refleje los tiempos de ejecución (en milisegundos) del módulo PythonA1.py, para los valores de n expuestos (10000, 20000, 40000, 80000, 160000, 320000, 640000 y 1280000). Si para alguna n tardara más de 60 segundos, ponga FdT ("Fuera de Tiempo"), tanto en este apartado como en los apartados posteriores.

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

Tamaño del problema (n)	Tiempo (milisegundos)
10000	1715
20000	7023
40000	28375
80000	FdT
160000	FdT
320000	FdT
640000	FdT
1280000	FdT

Haga una tabla en la que refleje, al menos para dos ordenadores a los que tenga acceso, los tiempos de ejecución (en milisegundos) del módulo PythonA1.py para los valores de n expuestos (10000, 20000, …, 640000 y 1280000). Referencie claramente para cada ordenador qué CPU es la existente y cantidad de Memoria principal RAM.

En los dos respectivos ordenadores:

Tamaño del problema (n)	Tiempo (milisegundos) Intel i5 12400 2.5GHz 16GB	Tiempo (milisegundos) AMD Ryzen 7 6800H 16GB
10000	1715	2531
20000	7023	9641
40000	28375	39247
80000	FdT	FdT
160000	FdT	FdT
320000	FdT	FdT
640000	FdT	FdT
1280000	FdT	FdT

Programe una clase de nombre JavaA1.java que calcule el problema de los números primos expuesto y que utilice el mismo algoritmo A1 para ver si un número es primo que utilizaba PythonA1.py. Posteriormente, hay que hacer una tabla en la que refleje los tiempos de ejecución (en milisegundos) SIN_OPTIMIZACIÓN de JavaA1.java, para los valores de n expuestos (10000, 20000, …, 640000 y 1280000). Para finalizar, compare esos tiempos con los obtenidos en Python (en un apartado anterior) para ese mismo algoritmo A1.

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

Tamaño del problema (n)	Tiempo (milisegundos) JavaA1 SIN Optimización	Relación PythonA1 / JavaA1
10000	374	4,59
20000	1507	4,66
40000	6012	4,72
80000	24025	-
160000	FdT	-
320000	FdT	-
640000	FdT	-
1280000	FdT	-

Haga una tabla en la que refleje los tiempos de ejecución (en milisegundos) de los módulos PythonA1.py, PythonA2.py y PythonA3.py, para los valores de n expuestos (10000, 20000, …, 640000 y 1280000).

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

Tamaño del problema (n)	Tiempo (milisegundos) PyhtonA1	Tiempo (milisegundos) PythonA2	Tiempo (milisegundos) PythonA3
10000	1715	207	101
20000	7023	783	382
40000	28375	2906	1435
80000	FdT	11008	5410
160000	FdT	41266	20259
320000	FdT	FdT	FdT
640000	FdT	FdT	FdT
1280000	FdT	FdT	FdT

Implemente esos mismos algoritmos A2 y A3 en Java, en dos clases de nombres respectivamente JavaA2.java y JavaA3.java.

Realice una tabla en la que refleje los tiempos de ejecución (en milisegundos), ejecutando “SIN_OPTIMIZACIÓN”, de las clases JavaA1.java, JavaA2.java y JavaA3.java, para los valores de n expuestos (10000, 20000, …, 640000 y 1280000).

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

Tamaño del problema (n)	Tiempo (milisegundos) JavaA1 SIN Optimización	Tiempo (milisegundos) JavaA2 SIN Optimización	Tiempo (milisegundos) JavaA3 SIN Optimización
10000	374	42	22

20000	1507	155	84
40000	6012	567	304
80000	24025	2103	1132
160000	FdT	7891	4268
320000	FdT	29874	16196
640000	FdT	FdT	60986
1280000	FdT	FdT	FdT

Realice una tabla en la que refleje los tiempos de ejecución (en milisegundos), ejecutando "CON_OPTIMIZACIÓN", de las clases JavaA1.java, JavaA2.java y JavaA3.java, para los valores de n expuestos (10000, 20000, …, 640000 y 1280000).

Esto medido con un ordenador de procesador Intel Core i5 12400 2.5GHz y 16GB de RAM:

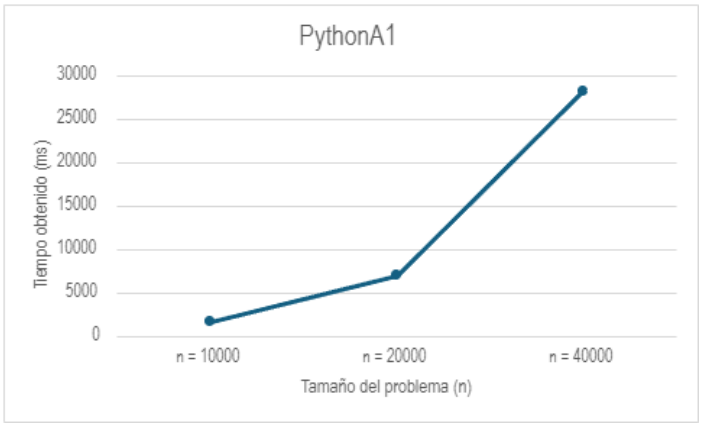
Tamaño del problema (n)	Tiempo (milisegundos) JavaA1 CON Optimización	Tiempo (milisegundos) JavaA2 CON Optimización	Tiempo (milisegundos) JavaA3 CON Optimización
10000	75	11	5
20000	292	32	15
40000	1157	114	60
80000	4632	421	213
160000	18482	1585	803
320000	FdT	5955	3003
640000	FdT	22648	11436
1280000	FdT	FdT	42810

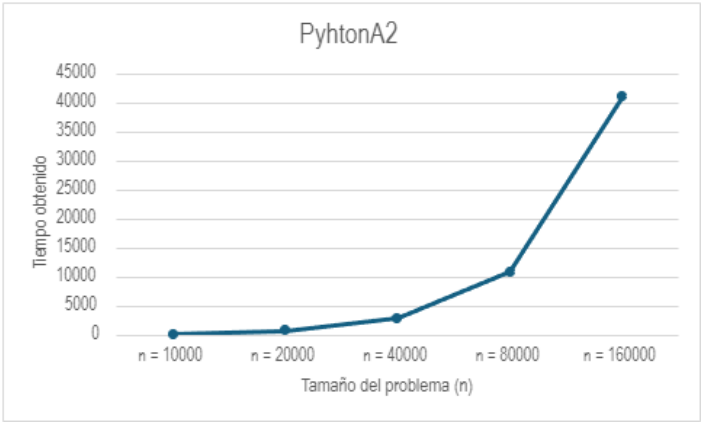
Finalmente, razone las conclusiones obtenidas comparando los tiempos antes obtenidos: con Python, con Java "SIN_OPTIMIZACIÓN y con Java "CON_OPTIMIZACIÓN".

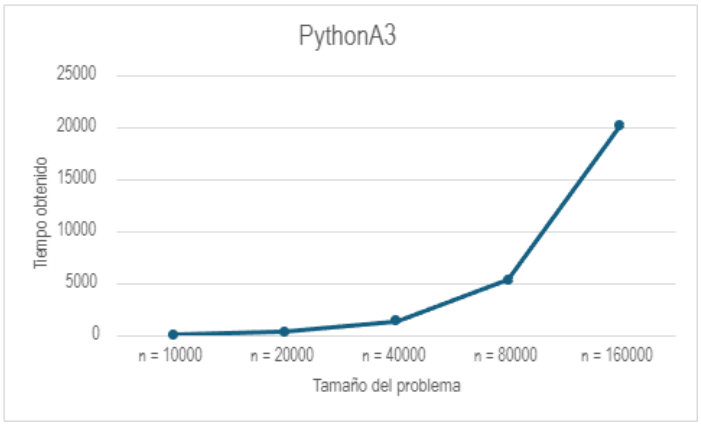
Como se puede apreciar por los resultados obtenidos, estos van acorde con lo que dicta el guion referente a cómo el entorno influye en el tiempo de ejecución, viendo cómo en Java los programas cuentan con un considerable mejor rendimiento que en Python (aunque en mi caso yo empleé Spyder para medir estos, que por lo que tengo entendido es peor que otras aplicaciones similares como Pyscripter, por lo que puede que eso haya inflado un poco la diferencia entre entorno), y al cómo el ordenador empleado también supone una gran influencia en esto.

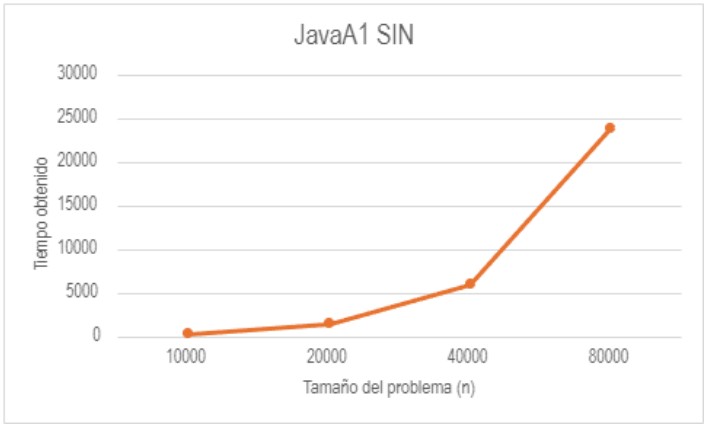
En cuanto a los programas, estos muestran que con menor complejidad los tiempos resultantes van de la mano a esta, y que al emplear la optimización los tiempos también se ven reducidos considerablemente.

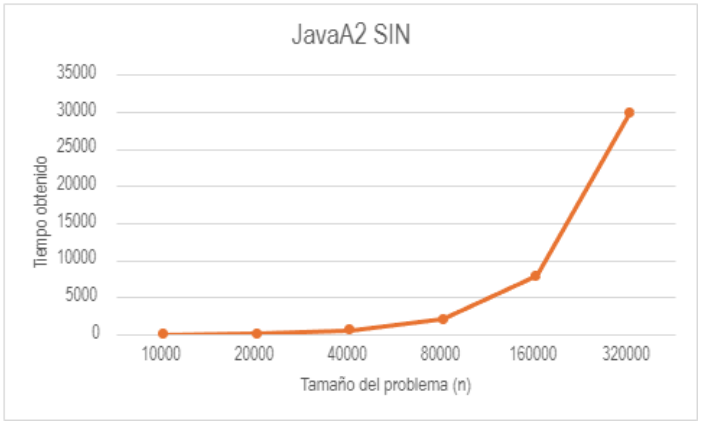
Sección de gráficas de los algoritmos empleados

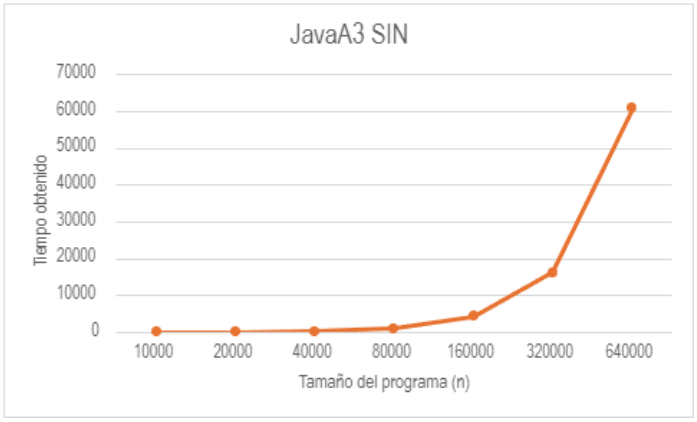


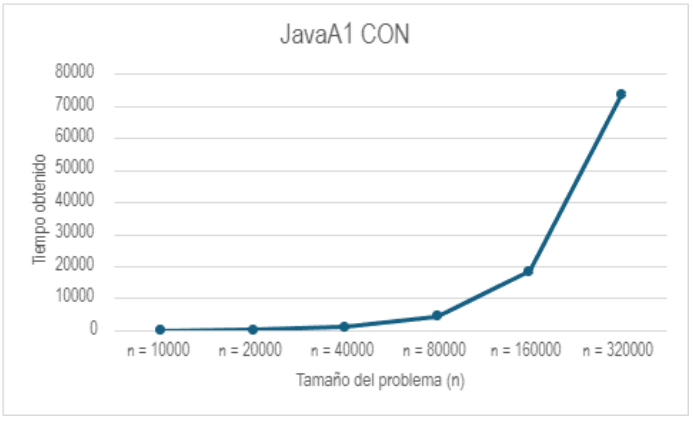


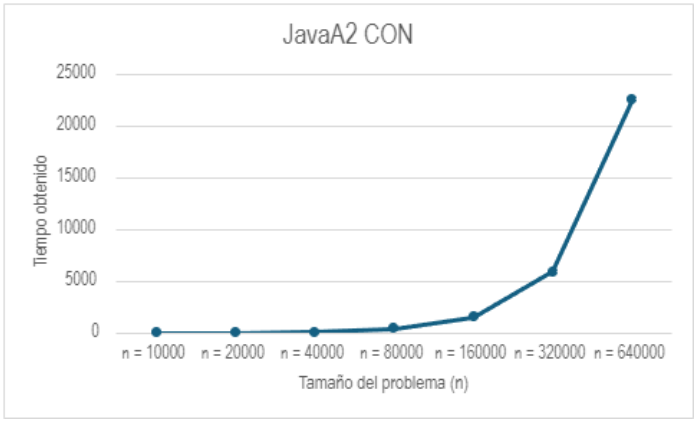


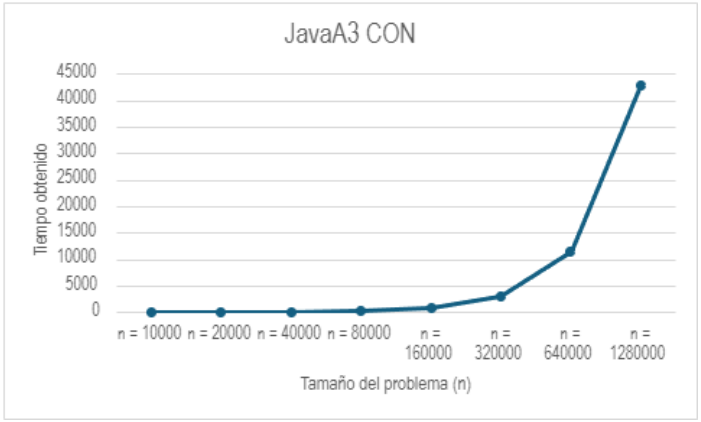

JavaA1 SIN
30000
25000
20000
15000
10000
5000
0
Tiempo obtenido
10000
20000
40000
80000
Tamaño del problema (n)


JavaA2 SIN
35000
30000
25000
20000
15000
10000
5000
0
Tiempo obtenido
10000
20000
40000
80000
160000
320000
Tamaño del problema (n)


JavaA3 SIN
70000
60000
50000
40000
30000
20000
10000
0
Tiempo obtenido
10000
20000
40000
80000
160000
320000
640000
Tamaño del programa (n)


JavaA1 CON
80000
70000
60000
50000
40000
30000
20000
10000
0
Tiempo obtenido
n = 10000
n = 20000
n = 40000
n = 80000
n = 160000
n = 320000
Tamaño del problema (n)


JavaA2 CON
25000
20000
15000
10000
5000
0
Tiempo obtenido
n = 10000
n = 20000
n = 40000
n = 80000
n = 160000
n = 320000
n = 640000
Tamaño del problema (n)


JavaA3 CON
45000
40000
35000
30000
25000
20000
15000
10000
5000
0
Tiempo obtenido
n = 10000
n = 20000
n = 40000
n = 80000
n = 160000
n = 320000
n = 640000
n = 1280000
Tamaño del problema (n)